

**Abstrakte Klassen und Methoden**

Als typisches Beispiel für eine Vererbungsrelation hatten wir schon die Klassen *Auto* und *Fahrrad* mit der gemeinsamen Superklasse *Fahrzeug* gesehen.

Wir hatten dies als Beispiel dafür genommen, dass gemeinsame Attribute (z. B. die Farbe des Fahrzeugs) oder Methoden (z. B. `fahren()`) durch die Superklasse abstrahiert werden können. In der Realität existieren allerdings nur die Subklassen – Autos, Fahrräder usw. Die Klasse *Fahrzeug* ist daher eine **abstrakte Klasse** und `fahren()` eine **abstrakte Methode**. Es können keine Objekte der Klasse *Fahrzeug* instanziiert werden und die Methode `fahren()` kann (auf der Superklasse) nicht aufgerufen werden.

**Aufgabe 1: Umsetzung in UML**

- a) Recherchieren Sie, wie man abstrakte Klassen und Methoden in UML-Diagrammen kennzeichnet.

- b) Entwerfen Sie ein UML-Diagramm zur folgenden Situation:

- Benachrichtigungen können entweder per E-Mail oder per SMS versendet werden.
- Eine Nachricht hat immer einen Empfänger.
- E-Mails werden von einer E-Mail-Adresse gesendet.
- SMS werden von einer Telefonnummer gesendet.



Aufgabe 2: Umsetzung in Python

- a) Finden Sie heraus, wie man abstrakte Klassen und Methoden in Python umsetzen kann.

- b) Implementieren Sie das UML-Diagramm aus Aufgabe 1 in Python.

- c) Was passiert, wenn keine Subklasse die abstrakte Methode der Superklasse implementiert?



Bonusaufgabe 3: Eigene Beispiele

- Finden Sie weitere Beispiele für Situationen, in denen die Verwendung abstrakter Klassen und Methoden sinnvoll wäre.
- Erstellen Sie UML-Diagramme für die Situationen.
- Setzen Sie die Diagramme in Python-Code um.